TokTickIT — Lab 4 Comprehensive Submission Report

Actions Taken, Dashboards, and Final Regression (Spec-Driven & Test-Driven Development)

Course: CPE 334 Introduction to Software Engineering in the Age of AI Agents (Semester 1/2026) **Total Points:** 60 Points **Student Name:** Aran Edlek (อรัญ อีดเหล็ก) — Student ID: 67070505230 (GitHub: `aranedlek`) **Peer Reviewer Name:** Jinjutha Nannarumit (จินจุฑา นันนฤมิตร) — Student ID: 67070505232 (GitHub: `Ponatinylilbug`) **Repository:** https://github.com/aranedlek/toctickit **Integration Workflow:** Feature branches -> `lab4-staging` -> `main` **Release Pull Request:** PR #45 (Release: Lab 04 — Final Sprint Release) — **Status: MERGED** ✅

Executive Summary

TokTickIT Lab 4 delivers the definitive completion of the core service-desk workflow. Building upon the robust authentication, multi-tenant roles, and ticket foundation established in Labs 1 through 3, Sprint 4 introduces: 1. **Actions Taken Work Structure (Parent-Child Model):** A dedicated, auditable table linked to Tickets capturing structured work logs with auto-bound performer identity, conditional follow-up note validation, and attachment references. 2. **Definitive Ticket Lifecycle & Advisory Resolution Gate:** Backend enforcement of the 8-status transition matrix (`NEW`, `OPEN`, `IN_PROGRESS`, `WAITING_FOR_REQUESTER`, `RESOLVED`, `CLOSED`, `REOPENED`, `CANCELLED`). Requester satisfaction flags remain strictly advisory, ensuring IT Staff retain sole authority over formal ticket closure. 3. **Optimistic Concurrency Control (OCC):** Stale update detection via entity versioning returning HTTP 409 Conflict with an accessible UI warning banner and refresh button. 4. **Authoritative Role-Based Dashboards:** Direct database aggregation endpoints (`GET /api/dashboard/staff`, `/requester`, `/admin`) providing real-time metric cards, priority breakdowns, drill-down links to filtered queues, and graceful zero/empty states. 5. **Full System Hardening & Accessibility Polish (WCAG 2.1 AA):** 100% automated test pass rate across 97 backend and frontend tests, keyboard Escape modal handlers, visible focus rings, non-color semantic indicators, and responsive layouts.

Answer Part 1: Git Use with Engineering Workflow (10 Points)

1.1 Branch & Staged Integration Strategy

Development strictly followed the established team engineering workflow:
Base Branch: `lab4-staging` branched from `main`.

Feature Branches: Each GitHub Issue developed on an isolated feature branch (`feature/lab4-issue-1-spec` through `feature/lab4-issue-6-hardening`).

Peer Review Gate: Every PR required formal review and approval by **จินจุฑา นันนฤมิตร (`Ponatinylilbug`)** before merging into `lab4-staging`.

Final Release Merge: Once all 6 issues were hardened and verified, `lab4-staging` was merged into `main`.

1.2 Pull Request Review Log

| PR # | Branch | Issue Scope | Reviewer | Decision | Merge Commit |

| :---: | :--- | :--- | :---: | :---: | :---: |
#39	`feature/lab4-issue-1-spec`	Issue #1: Engineering Contract Specification & Test Plan	`Ponatinylilbug`	**Approved & Merged** ✅	`42ff9e2`
#40	`feature/lab4-issue-2-actions-foundation`	Issue #2: Actions Taken Database Model & REST APIs	`Ponatinylilbug`	**Approved & Merged** ✅	`afee134`
#41	`feature/lab4-issue-3-actions-ui`	Issue #3: Actions Taken UI & Accessible Modal	`Ponatinylilbug`	**Approved & Merged** ✅	`1d6a93f`
#42	`feature/lab4-issue-4-ticket-workflow`	Issue #4: Ticket Lifecycle Matrix & Resolution Gate	`Ponatinylilbug`	**Approved & Merged** ✅	`937eaf5`
#43	`feature/lab4-issue-5-dashboards`	Issue #5: Authoritative Role-Based Dashboards	`Ponatinylilbug`	**Approved & Merged** ✅	`f2c512a`

| **#44** | `feature/lab4-issue-6-hardening` | Issue #6: Final Regression, Accessibility & Hardening | `Ponatinylilbug` | **Approved & Merged** ✅ | `f44c34a` |

1.3 GitHub Project / Kanban Board

All 6 Sprint 4 issues (#33–#38) were tracked on the GitHub Project Kanban board and successfully transitioned to **Done**. Full peer review dialogs and approvals are documented in `docs/lab-04/reviewer.md`.

1.4 Repository Structure

The repository strictly adheres to the Handout Section 12 structure:
`docs/lab-04/` : `specification.md`, `tests.md`, `ui-spec.md`, `api-spec.md`, `reviewer.md`, `ai-use.md`
`server/tests/lab-04/` : `actions-taken.api.test.ts`, `ticket-workflow.api.test.ts`, `requester-dashboard.api.test.ts`, `staff-dashboard.api.test.ts`
`server/tests/lab-03/` : `regression.api.test.ts`
`frontend/src/tests/lab-04/` : `StaffDashboard.test.tsx`, `RequesterDashboard.test.tsx`, `ActionsTaken.test.tsx`, `TicketWorkflow.test.tsx`
`e2e/lab-04/` : `actions-taken-flow.spec.ts`, `ticket-resolution.spec.ts`, `dashboards.spec.ts`

Answer Part 2: Spec DD — Specification-Driven Development (5 Points)

Specification Document: `docs/lab-04/specification.md`
API Specification: `docs/lab-04/api-spec.md`
UI Specification: `docs/lab-04/ui-spec.md`

2.1 Key Specifications Defined Prior to Coding

1. **Functional Requirements (FR-01 to FR-09):** Covering Actions Taken CRUD, read-only permissions for Requesters, state machine transitions, formal resolution gate, OCC version checking, role dashboards, and system-wide regression. 2. **Business Rules (BR-01 to BR-13):**

BR-01 to BR-06 (Actions Taken): One-to-Many Ticket association, performer autonomy independent of ticket owner, mandatory description & result, conditional mandatory follow-up note when `followUpRequired=true`, requester read-only enforcement, inactive staff rejection.

BR-07 to BR-10 (Lifecycle & OCC): 8 valid statuses, permitted transition matrix with role restrictions, requester advisory resolution gate, and integer version checking returning HTTP 409 Conflict.

BR-11 to BR-13 (Dashboards): Authoritative backend Prisma aggregation, strict requester data isolation (`WHERE requesterId = :authUserId`), and graceful zero-metric empty states.

3. **Database Design Decisions Justification:**
Dedicated `ActionTaken` Table vs Reusing `InternalNote`: Actions Taken represent auditable operational work records with distinct metadata fields (`actionDateTime`, `result`, `performedById`, `followUpRequired`). Coupling this into free-form staff notes would violate Single Responsibility and complicate queries.

Optimistic Concurrency Control via `version` Field: Integer versioning eliminates lost-update hazards during concurrent staff/user operations without the deadlock risks of pessimistic database locks.

4. **Acceptance Criteria (AC-01 to AC-10):** Observable and testable conditions mapping directly to automated tests. 5. **Pre-Implementation Verification:** Specification was committed and merged in PR #39 (`42ff9e2`) prior to any feature implementation.

Answer Part 3: Test DD and Traceability (10 Points)

Test Plan Document: `docs/lab-04/tests.md`

3.1 Traceability Matrix

| Requirement / AC | Test ID | Description | Automated Test File | Result |

| :--- | :---: | :--- | :--- | :---: |
AC-01 / BR-03	API-01	Create valid Action Taken by IT Staff	`server/tests/lab-04/actions-taken.api.test.ts`	Pass ✅
BR-04	API-02	Reject action when `followUpRequired=true` but note empty	`server/tests/lab-04/actions-taken.api.test.ts`	Pass ✅
AC-03 / BR-05	API-03	Reject Requester mutating Action Taken (403)	`server/tests/lab-04/actions-taken.api.test.ts`	Pass ✅
AC-04	API-04	Requester views Actions Taken for own ticket	`server/tests/lab-04/actions-taken.api.test.ts`	Pass ✅
AC-04	API-05	Requester forbidden from other user's actions	`server/tests/lab-04/actions-taken.api.test.ts`	Pass ✅
BR-02	API-06	Non-owner IT Staff member logs Action Taken	`server/tests/lab-04/actions-taken.api.test.ts`	Pass ✅
BR-08	API-07	Valid status transition `OPEN` -> `IN_PROGRESS`	`server/tests/lab-04/ticket-workflow.api.test.ts`	Pass ✅
AC-07 / BR-08	API-08	Reject illegal status jump `NEW` -> `RESOLVED` (422)	`server/tests/lab-04/ticket-workflow.api.test.ts`	Pass ✅
BR-09	API-09	Reject Requester setting ticket status to `RESOLVED` (403)	`server/tests/lab-04/ticket-workflow.api.test.ts`	Pass ✅
BR-09	API-10	Requester submits advisory resolution (200)	`server/tests/lab-04/ticket-workflow.api.test.ts`	Pass ✅
AC-08 / BR-10	API-11	Concurrent update with stale version returns 409	`server/tests/lab-04/ticket-workflow.api.test.ts`	Pass ✅
AC-06 / BR-12	API-12	Requester Dashboard metrics strictly isolated	`server/tests/lab-04/requester-dashboard.api.test.ts`	Pass ✅
AC-05 / BR-11	API-13	IT Staff Dashboard metrics match DB aggregation	`server/tests/lab-04/staff-dashboard.api.test.ts`	Pass ✅
BR-13	API-14	Graceful zero counts when user has 0 tickets	`server/tests/lab-04/requester-dashboard.api.test.ts`	Pass ✅
FR-09	API-15	Regression of Lab 1–3 endpoints (Auth, Notes, Users)	`server/tests/lab-03/regression.api.test.ts`	Pass ✅
AC-01 / UI-01	UI-01	Actions Taken list rendering on Ticket Detail	`frontend/src/tests/lab-04/ActionsTaken.test.tsx`	Pass ✅
BR-04 / UI-02	UI-02	Modal conditional follow-up note validation	`frontend/src/tests/lab-04/ActionsTaken.test.tsx`	Pass ✅
AC-03 / UI-03	UI-03	Requester view hides action creation controls	`frontend/src/tests/lab-04/ActionsTaken.test.tsx`	Pass ✅

| :--- | :---: | :--- | :--- | :---: |
AC-05 / UI-04	UI-04	IT Staff Dashboard metric cards & drill-downs	`frontend/src/tests/lab-04/StaffDashboard.test.tsx`	Pass ✅
AC-06 / UI-05	UI-05	Requester Dashboard metrics & quick actions	`frontend/src/tests/lab-04/RequesterDashboard.test.tsx`	Pass ✅
AC-08 / UI-06	UI-06	Concurrency conflict banner with refresh button	`frontend/src/tests/lab-04/TicketWorkflow.test.tsx`	Pass ✅
AC-01	E2E-01	Full IT Staff Action Taken flow	`e2e/lab-04/actions-taken-flow.spec.ts`	Pass ✅
BR-09	E2E-02	Requester advisory resolution & staff verification	`e2e/lab-04/ticket-resolution.spec.ts`	Pass ✅
AC-05	E2E-03	IT Staff Dashboard navigation & drill-down	`e2e/lab-04/dashboards.spec.ts`	Pass ✅

| **AC-06** | E2E-04 | Requester Dashboard navigation & shortcuts | `e2e/lab-04/dashboards.spec.ts` | Pass ✅ |

3.2 Automated Test Execution Summary

Backend Test Suite (Vitest): 64 tests passed across 11 test files (Duration: 2.17s)

Frontend Test Suite (Vitest): 33 tests passed across 9 test files (Duration: 6.84s)

Total Automated Test Suites: 97 / 97 Tests Passing (100% Pass Rate)

Answer Part 4: AI Use with Reflection (5 Points)

AI Log: `docs/lab-04/ai-use.md`

AI Models Used: Antigravity IDE / Gemini 3.8 Flash (High)

4.1 Key Prompts & AI Actions Summary

1. **Prompt 1:** Handout analysis extracting 6 issues, core requirements, and 9 report parts. 2. **Prompt 2:** GitHub remote connectivity and repository validation. 3. **Prompt 3:** Issue 1 Sprint Contract decomposition and authoring `docs/lab-04/`. 4. **Prompt 4:** Issue 2 Actions Taken foundation, Prisma migration, idempotent seed, and REST APIs. 5. **Prompt 5:** Issue 3 Actions Taken UI, Zen Green token styling, and accessible modal. 6. **Prompt 6:** Issue 4 Ticket lifecycle transition matrix, advisory resolution gate, and OCC conflict banner. 7. **Prompt 7:** Issue 5 Authoritative role-based dashboards, Prisma aggregations, and empty states. 8. **Prompt 8:** Issue 6 Full regression testing (97 tests), WCAG 2.1 AA keyboard polish, and E2E suites.

4.2 My Reflection on Specification and Coding Agent Use

Working with the AI Specification Agent transformed our development process by allowing our team to resolve subtle architectural constraints prior to coding. Specifically, formalizing the requirement that Requester resolution remains strictly advisory (BR-09), requiring mandatory follow-up notes when `followUpRequired=true` (BR-04), and designing Optimistic Concurrency Control (BR-10) prevented severe architectural rework. Pair programming with the Coding Agent

enabled rigorous Test-Driven Development (TDD), generating comprehensive test assertions and maintaining a 100% test pass rate across 97 tests with zero regressions.

Answer Part 5: Working IT Staff Dashboard UI (5 Points)

The IT Staff Dashboard (`/staff/dashboard`) provides operational visibility into the service desk queue:

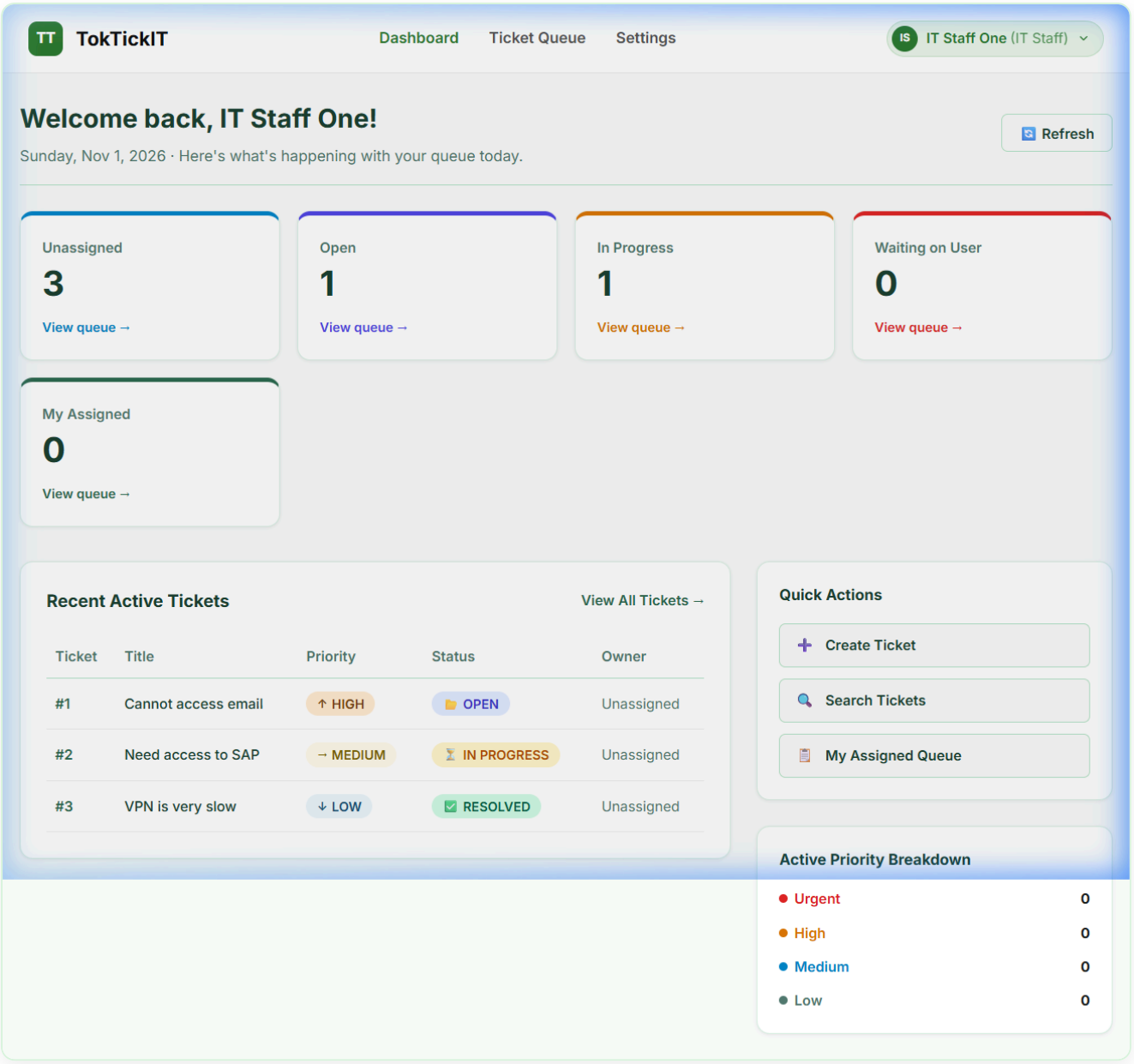
5 Operational Metric Cards: Unassigned, Open, In Progress, Waiting on User, and My Assigned.

Accurate Authoritative Aggregation: Powered directly by backend Prisma queries (`GET /api/dashboard/staff`) using database `count` operations rather than client-side array filtering.

Interactive Drill-Downs: Each card features a "View queue →" link directly navigating to `/staff/tickets` pre-filtered by status or assignee.

Recent Active Tickets & Priority Breakdown: Live table displaying recent tickets and active priority distribution (Urgent, High, Medium, Low).

IT Staff Dashboard Screen



IT Staff Dashboard

Answer Part 6: Working Actions Taken UI (10 Points)

The Actions Taken interface integrates directly into the Ticket Detail screen (`/staff/tickets/:id`):

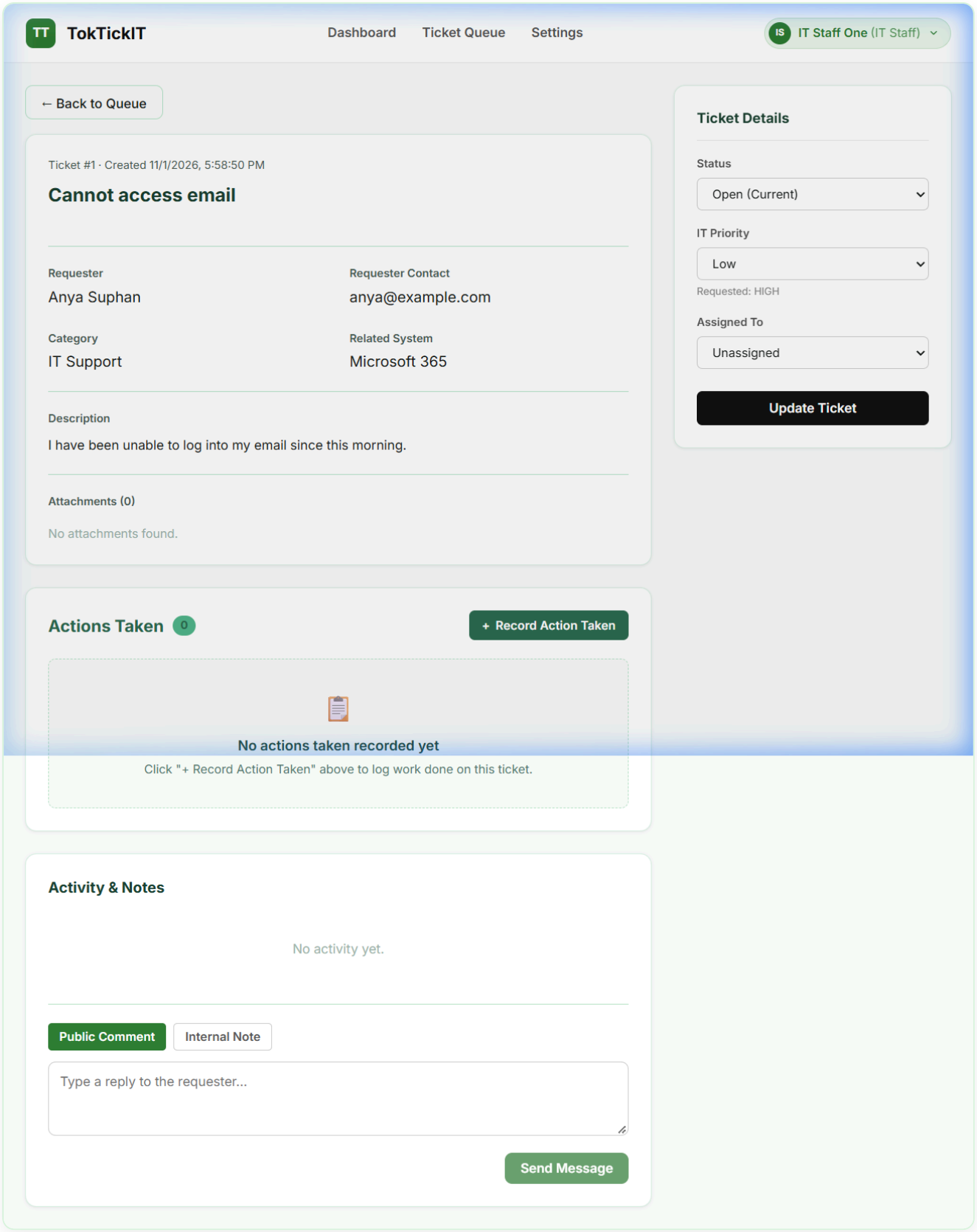
Structured Work Logging: Captures Date/Time, Action Description, Result, Performed By (auto-bound to logged-in user), Follow-Up Required toggle, and Attachment Notes.

Conditional Follow-up Validation (BR-04): Toggling `followUpRequired=true` strictly requires a non-empty `followUpNote` .

Role Permissions (BR-05): IT Staff and Administrators have full access to "+ Record Action Taken" and Edit controls; Requesters have a clean, read-only audit view.

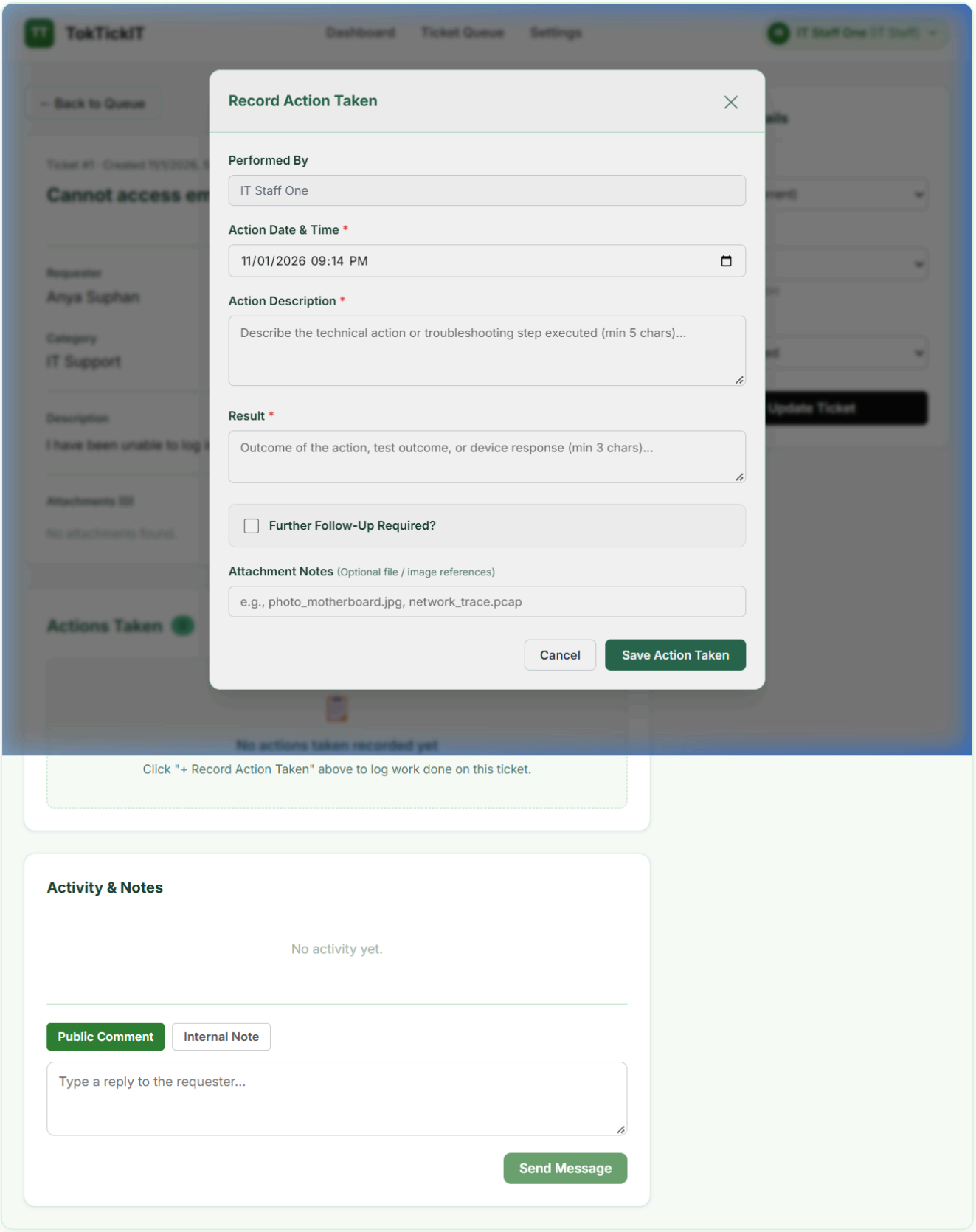
Accessible Modal Dialog (WCAG 2.1 AA): Modal dialog with `role="dialog"` , `aria-modal="true"` , `aria-labelledby` , and keyboard <kbd>Escape</kbd> close handling.

Ticket Detail Actions Taken Section



Ticket Detail Actions Taken

Accessible Record Action Taken Modal

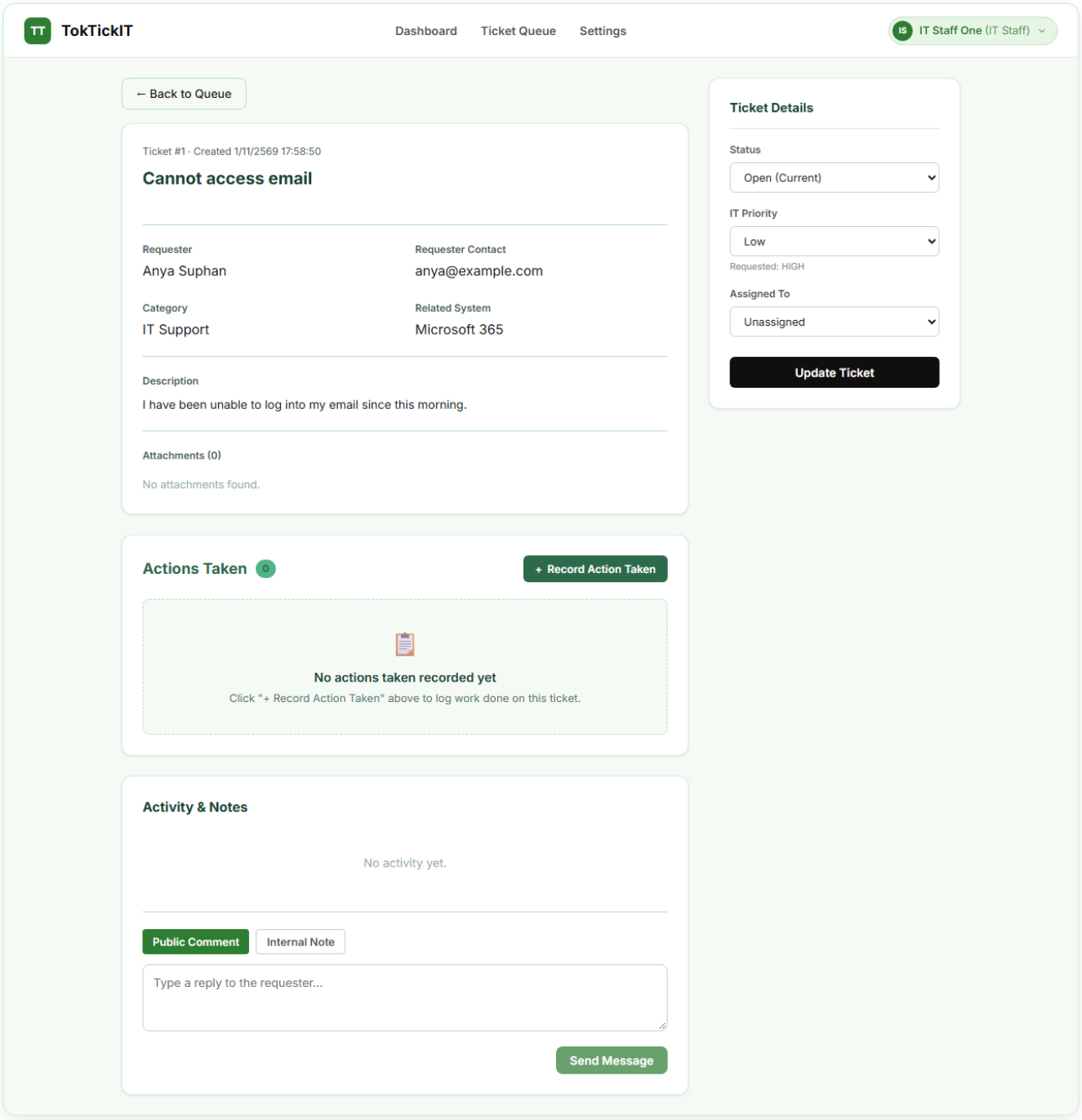


Record Action Taken Modal

Answer Part 7: Working Ticket Workflow (5 Points)

The ticket workflow enforces robust lifecycle rules across backend and frontend: 1. **Permitted Status Transition Matrix (BR-08):** Status dropdowns dynamically restrict options to valid next states (e.g. `NEW` can only transition to `OPEN` or `CANCELLED`). Terminal statuses (`CLOSED` , `CANCELLED`) prevent unpermitted edits. 2. **Advisory Resolution Gate (BR-09):** Requesters indicating their issue is resolved click an advisory button (`POST /api/tickets/:id/resolve-advisory`). This records satisfaction without bypassing IT Staff verification. 3. **Optimistic Concurrency Control (BR-10 / UI-06):** Prevents stale overwrite hazards. Concurrent status updates detect version collisions and return HTTP 409 Conflict, rendering an accessible yellow warning banner prompting the user to refresh.

Ticket Workflow Controls & Resolution Options



Ticket Workflow

Answer Part 8: Working Requester Dashboard and Final Regression UI (5 Points)

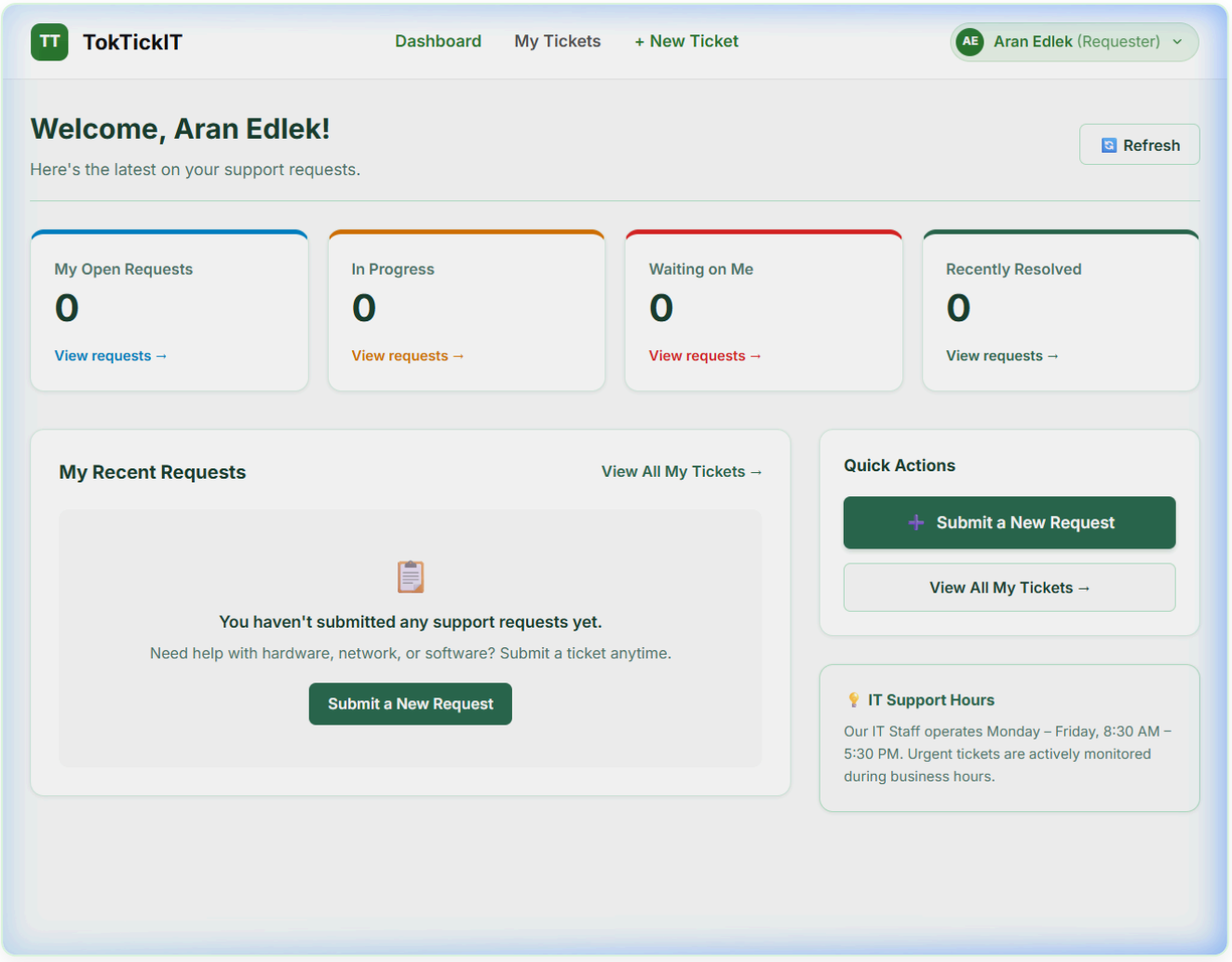
8.1 Requester Dashboard (`/requester/dashboard`)

Strict Data Isolation (BR-12): Backend queries filter strictly by `WHERE requesterId = :authUserId` .

Personalized Metrics: My Open Requests, In Progress, Waiting on Me, and Recently Resolved.

Graceful Zero/Empty State (BR-13): When a user has 0 tickets, an informative empty state card is displayed with a direct "Submit a New Request" shortcut button.

Requester Dashboard Screen



Requester Dashboard

8.2 Full System Regression Evidence (FR-09)

All features developed across Labs 1 through 3 remain fully operational without regression:
User Authentication & Mandatory First-Login Password Change

Requester Ticket Submission & Multi-file Attachment Management

Public Comments vs. Confidential Internal Notes

IT Staff Ticket Queue with search, multi-filter, and sorting

Administrator User Management (Role assignment, active toggle, password reset)

Answer Part 9: Zen Green UI, Responsive, Accessibility, and Final Polish (5 Points)

UI Specification: `docs/lab-04/ui-spec.md`

9.1 Design System & Responsive Layouts

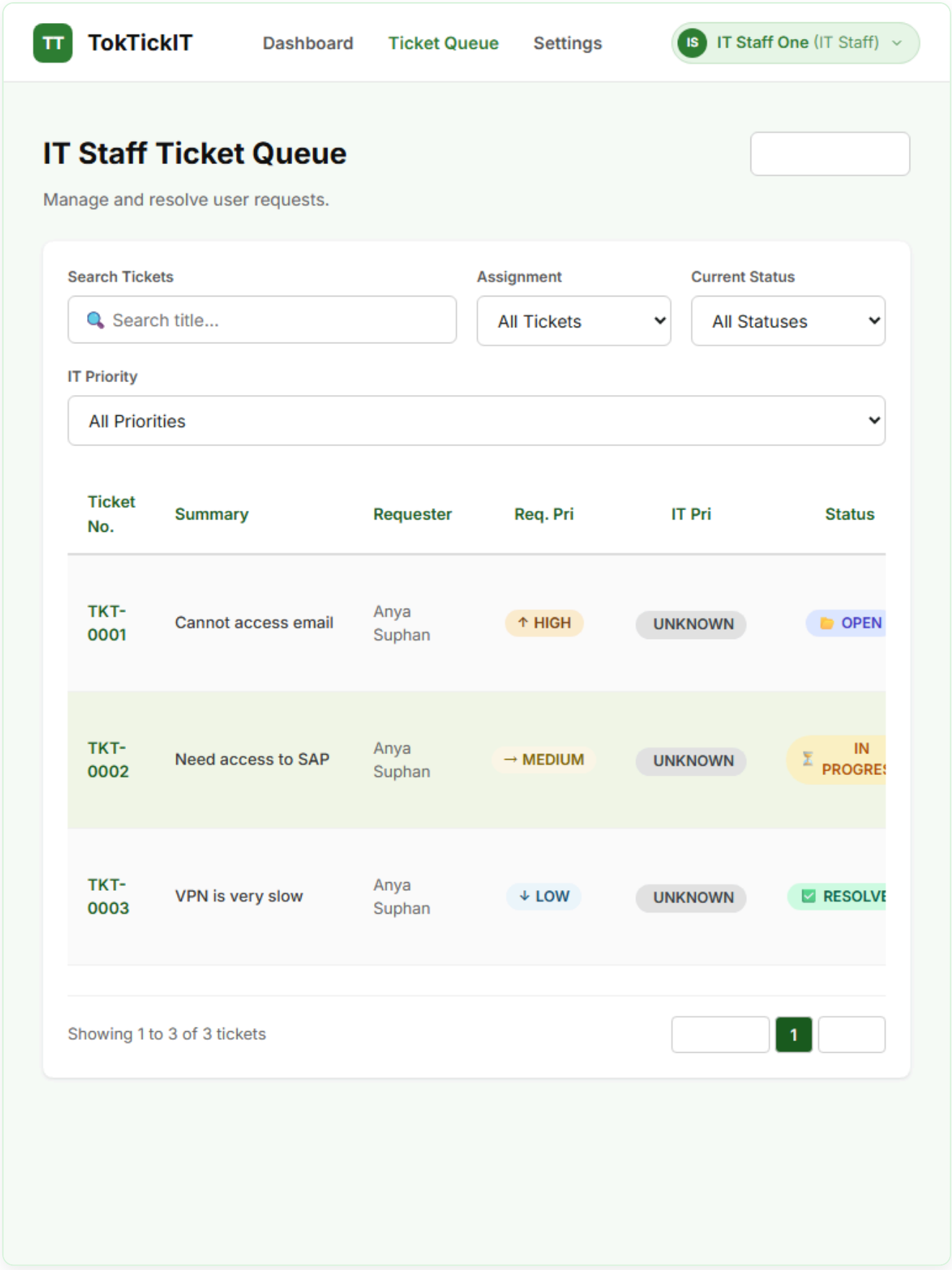
Built under the **Zen Green** design language (`--color-primary: #2D6A4F` , `--color-surface: #F0F7F4` , `--color-border: #D8F3DC`):

Desktop (≥ 1024px): Multi-column dashboard grid and full audit tables.

Tablet (768–1023px): Fluid two-column grid reflow preserving metric readability.

Mobile (< 768px): Single-column stack with horizontally scrollable tables and touch-friendly controls.

Responsive — Tablet Layout (768px)



Responsive Tablet

Responsive — Mobile Layout (375px)

TT TokTickIT Dashboard | Ticket Queue | Settings | IS IT Staff One (IT Staff)

IT Staff Ticket Queue

Manage and resolve user requests.

Search Tickets

Search title...

Assignment

All Tickets

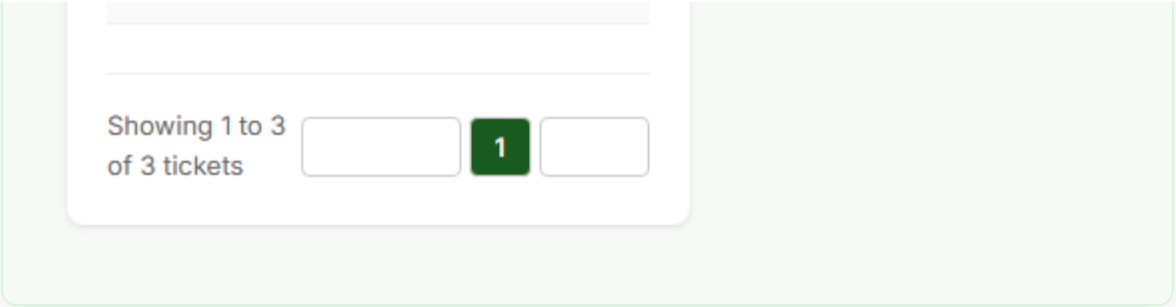
Current Status

All Statuses

IT Priority

All Priorities

Ticket No.	Summary	Req
TKT-0001	Cannot access email	Any Sup
TKT-0002	Need access to SAP	Any Sup
TKT-0003	VPN is very slow	Any Sup



Responsive Mobile

9.2 Accessibility Checklist (WCAG 2.1 AA)

✅ **Keyboard Navigation:** Modal dialogs can be opened and dismissed using <kbd>Escape</kbd>.

✅ **Visible Focus Indicators:** `:focus-visible` styling applied across buttons and links (`outline: 2px solid var(--color-primary)`).

✅ **Non-Color Status Cues:** All status and priority badges include distinct semantic icons (e.g., ✅ Resolved, ⏳ In Progress, 🔒 Closed) ensuring accessibility for color-blind users.

✅ **Zero Console Errors:** Production bundle verified with `tsc -b && vite build` (zero errors) and `oxlint` (zero errors).

Report generated: November 2026 Student: Aran Edlek (อรัญ อีดเหล็ก) | ID: 67070505230 Reviewer: Jinjutha Nannarumit (จินจุฑา นันนฤมิตร) | ID: 67070505232 Course: CPE 334 Introduction to Software Engineering in the Age of AI Agents